A full description of the syntax may be found at https://github.com/ruby/rbs/blob/master/docs/syntax.md; we'll talk about the most common usages here.

The .rbs file typically combines one or more entire modules into a single file, though, as with typical Ruby, there's nothing preventing you from splitting the file up as you please.

The basic structure of module and class declarations has the same syntax as regular Ruby, and has the side effect of allowing those constant names to be used as type names. In other words, the same way we use String as a type name, after declaring class Finder, we could also use Finder as a type in a method argument or return value or wherever.

Inside each class, this gem has two different kinds of declarations.

We declare attributes and constants, including VERSION: String. The Finder class declares @signatures: Hash[String, Array[String]], meaning it expects to have an instance variable called @signatures and the type of that variable is a Hash whose keys are of type String and whose values are of type Array[String]. The general syntax is the name of the instance variable, followed by a colon, followed by the type. The square bracket syntax here is called a *generic* (more on this in a bit), but it allows us to define both the type of a container and the type of objects in the container, so Array[String] is an Array container where each element is a String.

In general, any time you see a type in RBS, you can add a ? to the end of it to indicate that the value can be nil. So @name: String means the name has to be a string, but @name: String? means the name can be a string or can be nil.

The lines attr_reader dictionary: Array[String] and attr_reader words_to_find: Array[String] are also attribute declarations. A it does in Ruby itself, attr_reader is a shortcut for declaring the type of an instance variable, and a getter method. The related declaration attr_writer declares the type of the instance variable and the setter method, and attr_accessor declares all three. The syntax is the kind of declaration, the name of the attribute, a colon, and the type.

The rest of lines in the file are type signatures of methods. For example, def lookup: (String word) -> Array[String] tells us that the lookup method takes a positional argument named word of type String and returns an array of strings.

The general syntax here is def followed by the name of the method followed by a colon, followed by the attributes inside parentheses, followed by the -> arrow followed by the return type.

The attribute listing has a few variants. As that declaration shows, positional arguments have type first followed by the variable name—the name is actually optional, and is not checked against the name in the actual code. Keyword arguments, however, are in a different order, name, then colon, then type, so def lookup: (word: String) -> Array[String] would indicate that word is a keyword argument. Keyword arguments are checked against the actual Ruby method signature.

An optional argument is denoted with a ? prefix, so def lookup: (?String) is a method with an optional positional argument, but if the argument is specified, it can't be nil. The two kinds of optional can be combined: def lookup: (?String?) is a method that takes an argument that is both optional and can take a nil value.

Before we talk about more complex RBS syntax, let's take a look at how you can use RBS.